

Segunda entrega: Twitch analytics II

Grupo Heropass



Alumnos:

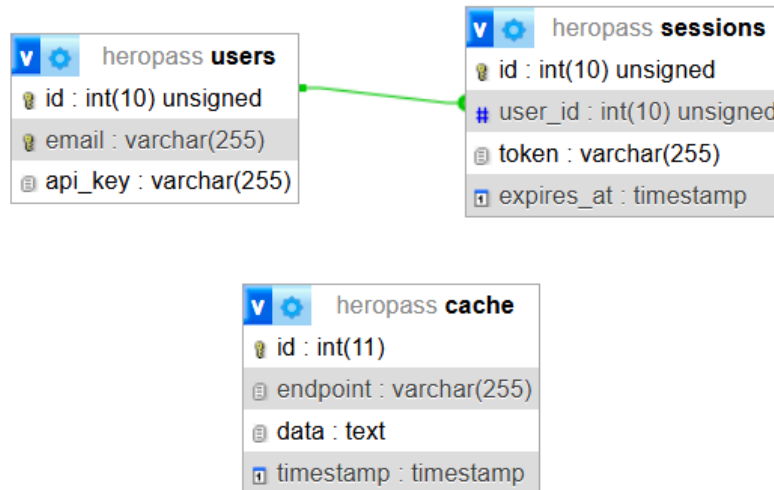
**Asier Muñoz, Iñigo Aznárez, Miguel Vallés y Alejandro
Molina**

ÍNDICE

Base de datos	3
1. Users	3
2. Sessions	3
3. Cache	3
Nuevo sistema de autenticación	4
1. Registro de Usuarios	4
2. Obtención de Token de Sesión	5
3. Uso de la Autenticación en el API	6
Nuevo Endpoint: "Top of the Tops" REVISAR	6
Códigos de respuesta y peticiones	7
Respuesta 200 OK	7
Caso Register	7
Caso Token	7
Caso Tops of the tops	8
Respuesta 400 Bad Request	8
Respuesta 401 Unauthorized	9
Respuesta 500 Internal Server Error	9
Problemas y decisiones	11

Base de datos

Para esta segunda entrega ha sido necesario crear una base de datos con las siguientes 3 tablas, una para cada nuevo endpoint:



1. Users

La tabla **users** nos permite tener un control de los usuarios que están registrados en nuestra API y el API key de cada usuario, que será necesario para generar un token de sesión.

2. Sessions

La tabla **sessions** almacena los tokens de autorización con el id del usuario al que pertenecen y una fecha de expiración de 3 días desde que se genera.

3. Cache

Con la tabla **cache** evitamos consultas excesivas a la API de Twitch, ya que se almacena la información de peticiones antiguas al endpoint *topsofthetops*. Si una petición es de este tipo y los registros de la base de datos son de hace menos de 10 minutos, devuelve los que estaban almacenados. Si por el contrario no lo son, los obtiene de la API de Twitch y actualiza la base de datos. Además si se quiere forzar la actualización de la base de datos se podrá realizar con el parámetro `since`.

Importante

Debido a que el servidor dónde tenemos alojada nuestra API no nos permite acceder desde un entorno local a su base de datos hemos exportado la estructura de la base de datos y la hemos publicado en el repositorio de GitHub.

Para poder ejecutar en un entorno local nuestra API habría que descargar el archivo `public_html/heropass.sql` e importarlo en el gestor de bases de datos local (phpMyAdmin en caso de usar xampp, el entorno local que utilizamos nosotros). Es importante que el nombre de la base de datos que se crea de manera local coincida con el del archivo `heropass.sql`, el nombre es `heropass`, ya que en el caso de que se cree con otro nombre no podría realizarse la conexión.

Nuevo sistema de autenticación

En esta segunda entrega vamos a mejorar la seguridad de nuestra API, implementando un sistema de autenticación basado en API Keys y token temporales.

1. Registro de Usuarios

Endpoint: POST /register

Este endpoint registra un usuario en la base de datos y genera una API key. Recibe una solicitud POST con un email en formato JSON, verifica su validez y lo busca en la base de datos. Si existe, actualiza su API key; si no, lo registra con una nueva, devolviendo siempre respuestas en JSON.

```
register.php

<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    echo json_encode(["error" => "Method not allowed"]);
    exit;
}

$data = json_decode(file_get_contents('php://input'), true);

if (!isset($data['email']) || !filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
    http_response_code(400);
    echo json_encode(["error" => "The email must be a valid email address"]);
    exit;
}

$email = $data['email'];
$api_key = bin2hex(random_bytes(16));

// Conectarse a bbdd ONLINE
//require_once 'bbdd/conexion.php';
// Conectarse a bbdd LOCAL
require_once 'bbdd/conexionLocal.php';

try {
    $pdo = new PDO("mysql:host=$host;dbname=$database", $user, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Verificar si el email ya existe
    $stmt = $pdo->prepare("SELECT * FROM users WHERE email = :email");
    $stmt->bindParam(':email', $email);
    $stmt->execute();

    if ($stmt->rowCount() > 0) { //rowCount() devuelve el número de filas afectadas por la última sentencia SQL
        // El email ya existe, actualizar api_key
        $stmt = $pdo->prepare("UPDATE users SET api_key = :api_key WHERE email = :email");
    } else {
        // El email no existe, insertar nuevo registro
        $stmt = $pdo->prepare("INSERT INTO users (email, api_key) VALUES (:email, :api_key)");
    }

    $stmt->bindParam(':email', $email);
    $stmt->bindParam(':api_key', $api_key);
    $stmt->execute();
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(["error" => "Database error: " . $e->getMessage()]);
    exit;
}

echo json_encode(["api_key" => $api_key]);
// cerrar la conexión
$conn->close();

?>
```

2. Obtención de Token de Sesión

Endpoint: POST /token

Este código recibe una solicitud POST con un email y una API Key en formato JSON. Verifica en la base de datos si ambos son válidos. Si no coinciden, responde con un error. Si son correctos, genera un token de sesión único con una validez de 3 días para poder realizar peticiones en nuestra API, lo guarda en la base de datos y lo devuelve en la respuesta.

```
token.php

try {
    $pdo = new PDO("mysql:host=$host;dbname=$database", $user, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Comprobar si el email y la API Key existen
    $stmt = $pdo->prepare("SELECT id FROM users WHERE email = :email AND api_key = :api_key");
    $stmt->bindParam(':email', $email);
    $stmt->bindParam(':api_key', $api_key);
    $stmt->execute();

    $user = $stmt->fetch(PDO::FETCH_ASSOC);
    if (!$user) {
        http_response_code(401);
        echo json_encode(["error" => "Invalid email or api_key"]);
        exit;
    }

    // Generar un token de sesión
    $token = bin2hex(random_bytes(16));
    $expiration = date('Y-m-d H:i:s', time() + (3 * 24 * 60 * 60)); // 3 días

    // Guardar el token en la base de datos
    $save_stmt = $pdo->prepare("INSERT INTO sessions (user_id, token, expires_at) VALUES (:user_id, :token, :expires_at)");
    $save_stmt->bindParam(':user_id', $user['id']);
    $save_stmt->bindParam(':token', $token);
    $save_stmt->bindParam(':expires_at', $expiration);
    $save_stmt->execute();

    echo json_encode(["token" => $token]);
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(["error" => "Database error: " . $e->getMessage()]);
    exit;
}

// Función para verificar el token
function verificarToken($token) {
    require 'bbdd/conexion.php';

    try {
        $pdo = new PDO("mysql:host=$host;dbname=$database", $user, $password);
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        $stmt = $pdo->prepare("SELECT * FROM sessions WHERE token = :token AND expires_at > NOW()");
        $stmt->bindParam(':token', $token);
        $stmt->execute();

        $session = $stmt->fetch(PDO::FETCH_ASSOC);
        if ($session) {
            return $session['user_id'];
        } else {
            return false;
        }
    } catch (PDOException $e) {
        return false;
    }
}
```

3. Uso de la Autenticación en el API

Este endpoints incluye autenticación por token, requiriendo que cada petición envíe el token de sesión en el encabezado Authorization. Esto garantiza el acceso solo a usuarios registrados, controla el ciclo de vida de los tokens y mejora la seguridad.

Para ésta versión hemos utilizado de encabezado de autorización X-Auth-Token ya que el servidor donde tenemos la API no nos permite utilizar Authorization Bearer Token.

Nuevo Endpoint: "Top of the Tops"

URL: `/analytics/topsofthetops`

El endpoint obtiene los 40 videos más vistos de los tres juegos más populares en Twitch, usando un token de autenticación. Primero, verifica si los datos están en caché (máximo 10 minutos) y los devuelve si son válidos. Si no, consulta la API de Twitch, procesa la información, identifica al creador con más visualizaciones y guarda los datos. También permite forzar una actualización con el parámetro `since` y maneja errores de la API o la base de datos.

Para éste script hemos decidido no incluir captura de pantalla del código ya que es más extenso que el resto y no consideramos necesario incluirla en la memoria. Para su visualización puede acceder al repositorio de github al directorio:

`public_html/analytics/topsofthetops.php`

Códigos de respuesta y peticiones

Respuesta 200 OK

Caso Register

The screenshot shows a REST client interface with a POST request to `http://localhost/Clase/V&VSoftware/HEROPASS.github.io/public_html/register`. The request body is a JSON object: `{ "email": "usuario@example.com" }`. The response status is 200 OK, with a time of 44 ms and a size of 324 B. The response body is a JSON object: `{ "api_key": "a984d03a60160f0bfbdb66a724f9f9fe9" }`.

```
POST http://localhost/Clase/V&VSoftware/HEROPASS.github.io/public_html/register

{
  "email": "usuario@example.com"
}
```

Status: 200 OK Time: 44 ms Size: 324 B

```
{
  "api_key": "a984d03a60160f0bfbdb66a724f9f9fe9"
}
```

Caso Token

The screenshot shows a REST client interface with a POST request to `http://localhost/Clase/V&VSoftware/HEROPASS.github.io/public_html/token`. The request body is a JSON object: `{ "email": "usuario@example.com", "api_key": "feafdd090723075095fcb1dd02eb36fd" }`. The response status is 200 OK, with a time of 81 ms and a size of 322 B. The response body is a JSON object: `{ "token": "6afeb6241a21312a9800b9e702135714" }`.

```
POST http://localhost/Clase/V&VSoftware/HEROPASS.github.io/public_html/token

{
  "email": "usuario@example.com",
  "api_key": "feafdd090723075095fcb1dd02eb36fd"
}
```

Status: 200 OK Time: 81 ms Size: 322 B

```
{
  "token": "6afeb6241a21312a9800b9e702135714"
}
```


Caso Tops of the tops

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost/Clase/V&VSoftware/HEROPASS.github.io/public_html/analytics/topsofthetops/`
- Authorization:** API Key
- Key:** X-Auth-Token
- Value:** 183db20f38bb28361c1c2d5b99a42664
- Add To:** Headers
- Status:** 200 OK
- Time:** 3.82 s
- Size:** 1.57 KB
- Body (JSON):**

```
1 {
2   "game_id": "509658",
3   "game_name": "Just Chatting",
4   "user_name": "KaiCenat",
5   "total_videos": 36,
6   "total_views": 414862225,
7   "most_viewed_title": "🇲🇵 MAFIATHON 2 🇲🇵 KAI X KEVIN HART X DRUSKI 🇲🇵 DAY 27 🇲🇵 20% OF REVENUE GOING TO SCHOOL IN NIGERIA 🇲🇵 ALL
8   MONTH 🇲🇵 CLICK HERE 🇲🇵 ISubathon",
9   "most_viewed_views": 24869844,
10  "most_viewed_duration": "22h5m32s",
11  "most_viewed_created_at": "2024-11-28T02:06:07Z"
12 },
13 {
14   "game_id": "21779",
15   "game_name": "League of Legends",
16   "user_name": "Riot Games",
```

Respuesta 400 Bad Request

The screenshot shows a REST client interface with the following details:

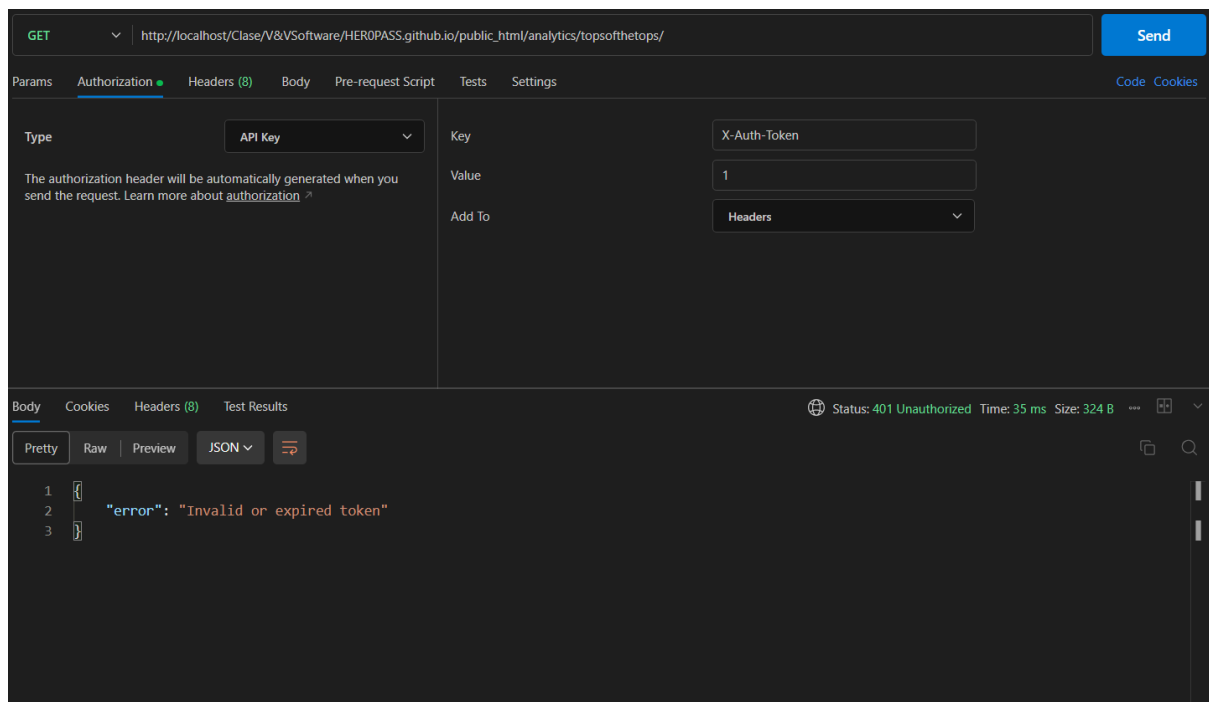
- Method:** POST
- URL:** `http://localhost/Clase/V&VSoftware/HEROPASS.github.io/public_html/register`
- Body:** raw
- Body Content:**

```
1 {
2   "email": "123"
3 }
```
- Status:** 400 Bad Request
- Time:** 15 ms
- Size:** 301 B
- Body (JSON):**

```
1 {
2   "error": "The email must be a valid email address"
3 }
```

En este caso el endpoint register recibe un email inválido y devuelve esta respuesta.

Respuesta 401 Unauthorized

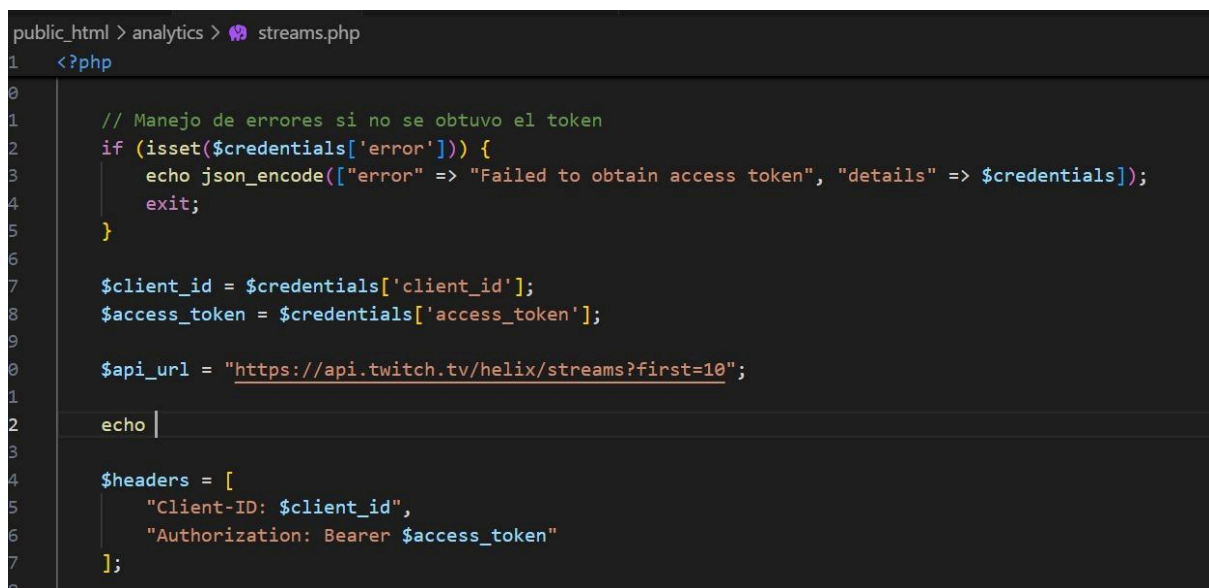


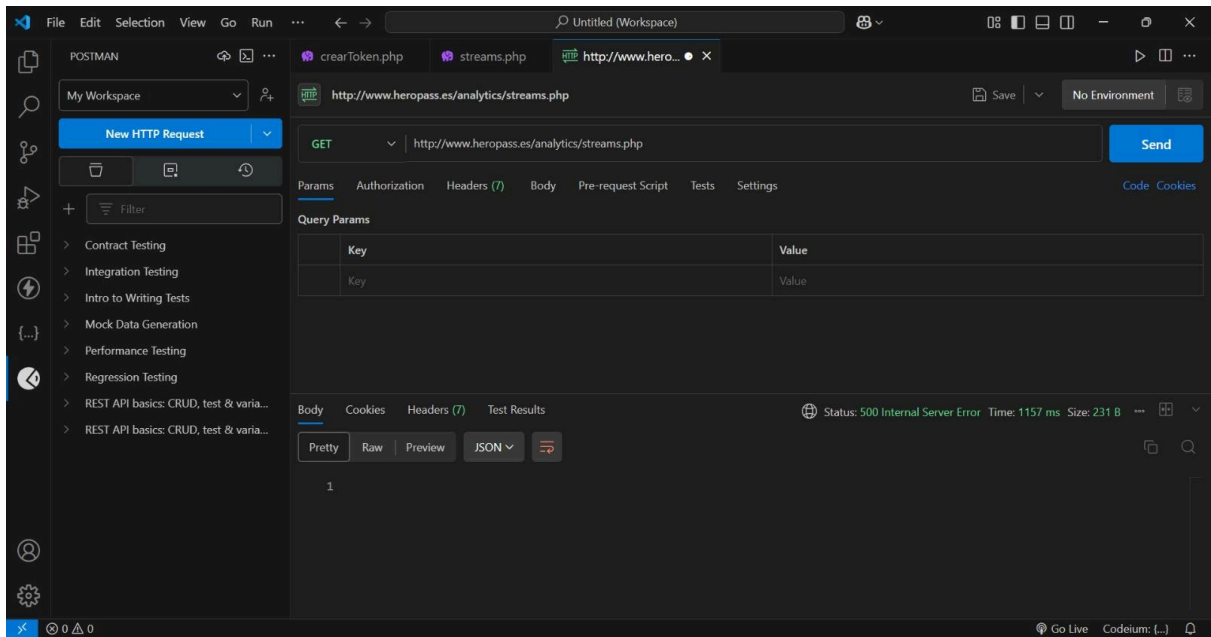
En este caso el endpoint [topsofthetops](#) recibe un token inválido y devuelve esta respuesta.

Respuesta 404 Not Found

En principio, según nuestra implementación siempre debería haber datos en la base de datos, o la API debería ser capaz de consultarlos de la API de twitch por lo que esta respuesta no debería darse.

Respuesta 500 Internal Server Error





Como en la entrega anterior, en principio esta respuesta no debería darse a no ser de un fallo en el servidor, en este caso lo hemos forzado con un error en el código y es que hay un echo sin cerrar que produce este error.

Problemas y decisiones

Problemas con IONOS:

- Authorization Bearer token no disponible:

Cuando empezamos a trabajar con el nuevo endpoint *topsofthetops*, nos encontramos con que a la hora de sacar el producto a producción, el servicio de alojamiento que estamos utilizando no nos permitía el uso del header de autorización. Lo que hemos decidido hacer es cambiarle el nombre a 'X-Auth-Token' y de esta forma poder pasar el token correctamente.

- Acceso a la base de datos restringido:

A la hora de trabajar en local hemos tenido problemas con la base de datos que nos proporciona IONOS, ya que este solo permite conexiones en línea y no desde local, por lo que hemos tenido que preparar nuestro propio entorno con XAMPP, e ir exportando la base de datos para poder utilizarla.

En las próximas entregas consideraremos migrar de entorno y comenzar a usar Heroku.

Problemas con el .htaccess:

Este archivo nos dio varios problemas, si conseguimos que en producción el archivo funcionara correctamente, en local dejaba de funcionar, y si lo cambiábamos y conseguimos que en local funcionase, a la hora de llevarlo a producción no funcionaba. Lo que decidimos al final fue tener 1 archivo distinto para trabajar en local y otro para tenerlo en producción.